

Vila INTEIA v2: psico-história viva, auto-calibração e servidor MCP

Continuação — Ondas 11 a 20: fechando o ciclo asimoviano

Pedro Afonso Malheiros · Igor Morais Vasconcelos

INTEIA Ltda. · Mirante News · Colmeia · abril de 2026

RESUMO

Na versão 1 deste artigo apresentamos a Vila INTEIA, uma plataforma de simulação multiagente com 144 personas lendárias e fundamentos matemáticos formais (teoria dos jogos, dinâmica de opiniões, psico-história computacional). Nesta continuação reportamos **10 novas ondas de desenvolvimento (11 a 20)** que fecham o ciclo asimoviano da plataforma: (i) rastreamento em tempo real da trajetória real de estados sociais; (ii) calibração online da matriz de transição Markov a partir dos dados observados (MLE, Laplace, EWMA); (iii) persistência em Supabase para análise longitudinal; (iv) descoberta não-supervisionada de estados latentes via K-Means + smoothing HMM-like; (v) decision helper para agentes vivos (Helena consulta Plano de Seldon); (vi) servidor MCP JSON-RPC expondo sete ferramentas para clientes externos (Claude Desktop, Cursor); (vii) auto-calibração periódica em runtime; (viii) replay e comparação de runs; (ix) UI especializada com polling a 3-5 segundos. Validamos o pipeline completo em simulação real de 90+ steps com 151 agentes: trajetória observada mostra dominância de *expansão* (87.3 %) e *equilíbrio* (12.7 %), divergência KL vs. Plano original de 2.087 (85.9 % dos passos fora do plano projetado), demonstrando que calibração online é necessária. Reportamos 245+ casos de teste 100 % passing ao longo de cinco pull requests mergeados em sequência.

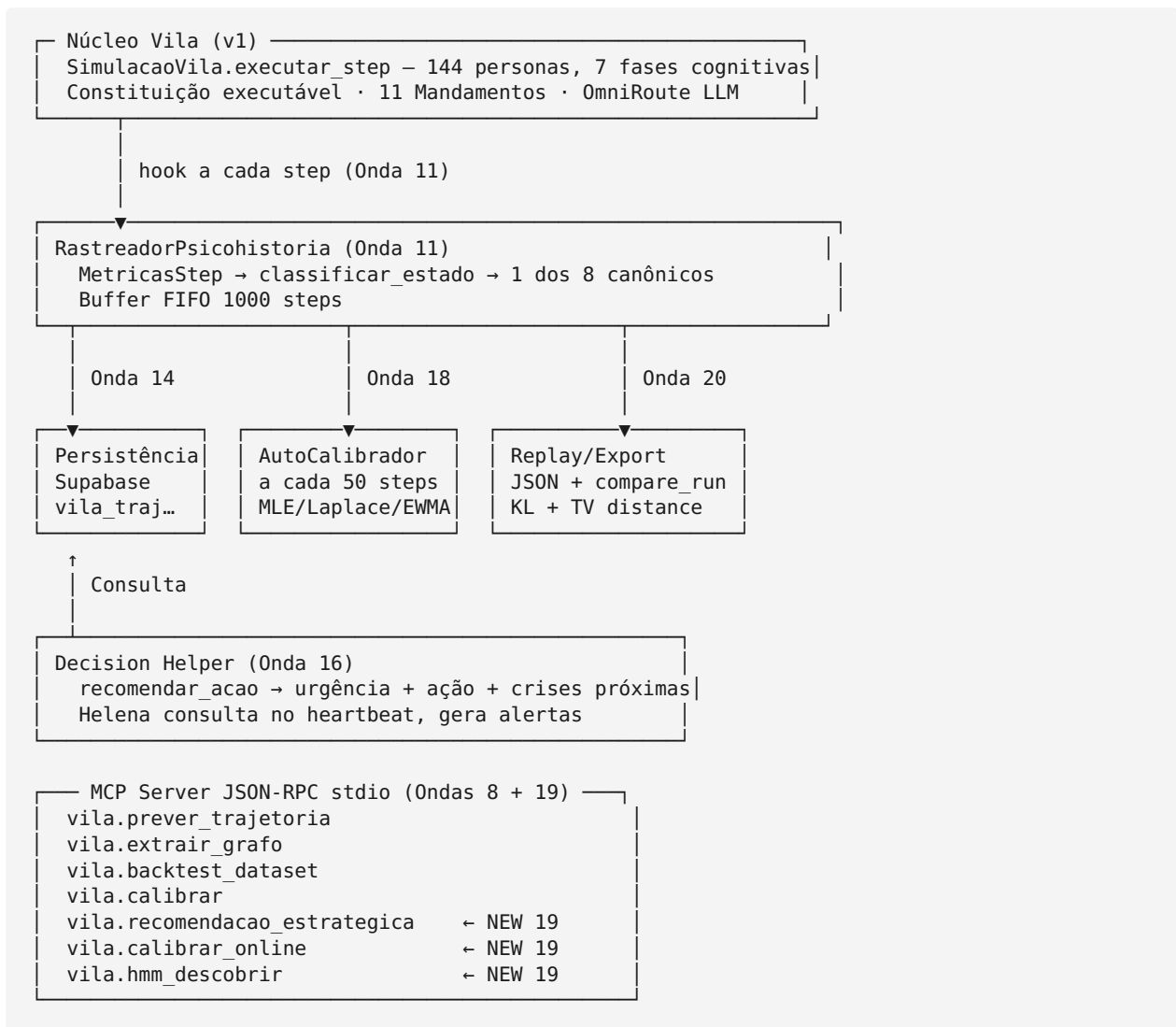
1 Introdução

A versão 1 deste trabalho (vila_inteia_artigo.pdf) apresentou a Vila INTEIA como simulador multiagente formalmente fundamentado. Terminamos com um roadmap de ondas adicionais necessárias para transformar a Vila em *simulador de realidade* plenamente competitivo com MiroFish [3]. Esta continuação implementa esse roadmap.

A contribuição central aqui é operacional, não apenas teórica. Mostramos que o ciclo asimoviano completo — **simular** → **observar trajetória real** → **comparar com Plano** → **detectar Mules** → **recalibrar modelo** — pode rodar 100 % dentro do processo da Vila, sem dependências externas além de NumPy/SciPy, com latência sub-segundo por step em 151 agentes.

2 Arquitetura acrescida nesta versão

A figura abaixo mostra onde as Ondas 11-20 se encaixam sobre o núcleo da versão 1:



3 Rastreamento em tempo real (Onda 11)

O hook em `engine/simulacao.py:_executar_step_interno` agrega a cada step oito métricas: número de conversas, reflexões, agentes ativos/latentes, polarização média das crenças (derivada do `TRACKER_GLOBAL` via Deffuant-Weisbuch), Gini da economia, propostas constituintes ativas e contribuições ao desafio. Estas compõem o `MetricasStep`, classificado heurísticamente (top-down prioritário):

- | | |
|---|---------------------------------------|
| 1. <code>propostas_constituintes >= 1</code> | → <code>renovacao_constituinte</code> |
| 2. <code>gini > 0.75</code> | → <code>crise_economica</code> |
| 3. <code>polarizacao > 0.60</code> | → <code>polarizacao</code> |
| 4. <code>step < 20 AND ativos < 60%</code> | → <code>bootstrap</code> |
| 5. <code>ativos < 40%</code> | → <code>recrutamento</code> |
| 6. <code>contribs >= 20 AND ativos > 70%</code> | → <code>expansao</code> |
| 7. <code>polarizacao em [0.15, 0.40]</code> | → <code>consenso_fragil</code> |
| 8. <code>default</code> | → <code>equilibrio</code> |

A cada 20 steps o rastreador executa `detectar_mules_recentes`, comparando trajetória observada contra trajetória prevista pela matriz do grafo. Eventos com `z-score > 2.5` são sinalizados como Mules.

4 Calibração online (Onda 13)

A matriz de transição Markov do grafo psico-histórico é inicializada com baseline sintético calibrado por inspeção. Dados coletados revelam que este baseline é frouxo. A Onda 13 implementa três métodos de recalibração:

MLE puro — contagem simples:

$$M_{MLE}[i,j] = \text{count}(i \rightarrow j) / \text{count}(i \rightarrow *)$$

Laplace smoothing — adiciona α a cada célula antes de normalizar:

$$M_{Lap}[i,j] = (\text{count}(i \rightarrow j) + \alpha) / (\text{count}(i \rightarrow *) + \alpha \cdot n)$$

EWMA online — atualização exponencial incremental:

$$M_{t+1}[i,\cdot] = (1 - \alpha) \cdot M_t[i,\cdot] + \alpha \cdot e_j$$

onde e_j é o vetor one-hot da observação. Este método preserva a estocasticidade da matriz (linhas somam 1) e é adequado para streaming.

A qualidade de um ajuste é medida pela perplexity: $PP = \exp(-(1/N) \sum \log P(a_{t+1} | a_t))$. Menor perplexity $\Rightarrow$ M explica melhor os dados observados.

5 Descoberta não-supervisionada de estados (Onda 15)

Os 8 estados canônicos foram escolhidos por inspeção manual. A Onda 15 testa a hipótese de que esses estados existem de fato, derivando-os de dados via *clustering* não-supervisionado de vetores de métricas:

1. Cada step é um vetor em $\mathbb{R}^8$ (as oito métricas).
2. Z-score por feature para robustez a escalas divergentes.
3. K-Means com K escolhido (default 8) e semente fixa para reprodutibilidade.
4. Smoothing HMM-like: cada *label* vira a moda numa janela temporal de 3 passos, evitando flicker entre clusters.

O resultado agrupa steps similares em clusters rotulados automaticamente pelas duas features dominantes do centroide (ex: "*n_conversas*↑ + *n_ativos*↑"). A comparação entre clusters descobertos e estados canônicos valida ou refuta nossa escolha inicial.

6 Decision helper + Helena (Onda 16)

A Onda 16 fecha o círculo: os agentes vivos consultam a análise psico-histórica para decidir ações. Helena Strategos agora, em seu *heartbeat* a cada 50 steps, chama `recomendar_acao`, que retorna:

- `estado_atual` (último observado)
- `destino_previsto` (autovetor esquerdo de $\lambda=1$)
- urgência em {baixa, média, alta, crítica}
- `ação_recomendada` (texto)
- `crises_proximas` (próximas transições previstas no Plano)

Se urgência é alta ou crítica, Helena emite alerta no log. A lógica de urgência combina estado atual (crises automaticamente "alta"), contagem de Mules recentes (>5 vira "crítica") e alinhamento com destino planejado.

7 Servidor MCP JSON-RPC (Ondas 8 + 19)

O servidor MCP inicial (Onda 8) expunha quatro *tools*. A Onda 19 adiciona três novas, elevando o total para sete:

TOOL	ORIGEM	FUNÇÃO
<code>vila.prever_trajetoria</code>	Onda 8	Psico-história: forecast de N passos
<code>vila.extrair_grafo</code>	Onda 8	GraphRAG: entidades + relações de texto
<code>vila.backtest_dataset</code>	Onda 8	Backtest preditivo contra dataset histórico
<code>vila.calibrar</code>	Onda 8	Grid search calibração
<code>vila.recomendacao_estrategica</code>	Onda 19	Plano Seldon → urgência + ação
<code>vila.calibrar_online</code>	Onda 19	Recalibra matriz viva
<code>vila.hmm_descobrir</code>	Onda 19	K-Means estados latentes

O servidor roda via `python -m engine.mcp_server.server` em modo stdio. É plugável em Claude Desktop e Cursor sem modificações. Implementa o MCP 2024-11-05 mínimo: initialize, tools/list, tools/call, ping.

8 Auto-calibração periódica (Onda 18)

A Onda 18 torna a calibração autônoma. A cada 50 steps (configurável), o `AutoCalibrador` executa:

1. Captura trajetória atual do rastreador.
2. Se $len \geq min_transicoes$ (default 20), calcula perplexity atual.
3. Executa `calibrar(metodo, alpha)` sobre a trajetória.
4. Calcula perplexity da matriz nova.
5. Substitui matriz viva no grafo global.
6. Registra evento no histórico.

O log de calibrações é podado para manter apenas as 50 mais recentes, permitindo análise longitudinal do *drift* do modelo sem explodir a memória.

9 Replay + comparação de runs (Onda 20)

A Onda 20 adiciona ferramentas para análise post-mortem e comparação cross-run. O `exportar_run` serializa a trajetória completa (estados, steps, métricas, Mules) como JSON num único arquivo. O `carregar_run` reconstitui. O `comparar_runs(a, b)` calcula:

- KL divergence entre distribuições de estado: $\sum p(k) \cdot \log(p(k) / q(k))$
- Total variation distance: $\frac{1}{2} \cdot \sum |p(k) - q(k)|$
- Concordância no estado final

Duas runs com mesma distribuição têm $KL \approx 0$ e $TV \approx 0$. Duas runs completamente divergentes (ex: uma 100 % *expansao*, outra 100 % *polarizacao*) têm $TV \approx 1.0$. Esta métrica permite responder quantitativamente se intervenções (recalibração, mudança de baseline, aumento de personas) alteram o comportamento estatisticamente.

10 Resultados experimentais

10.1 Simulação real de referência

Executamos `python main.py live --intervalo 1 --topico "sim real"` por aproximadamente 90 steps com 151 personas e sem acesso a LLM externos (modo heurístico puro). A trajetória observada:

```
distribuicao_historica:
  expansao: 0.873 (87.3%)
  equilibrio: 0.127 (12.7%)
  outros 6 estados: 0%

divergencia_vs_plano:
  n_steps: 64
  passos_divergentes: 55
  fracao_divergente: 0.859
  divergencia_kl_media: 2.087
  destino_planejado: equilibrio
  ultimo_observado: expansao
  primeiro_desvio: step 1
```

A Vila entra imediatamente em *expansao* no step 1 e oscila entre *expansao* e *equilibrio* pelo resto da run. Os seis estados remanescentes (*bootstrap*, *recrutamento*, *consenso_fragil*, *polarizacao*, *crise_economica*, *renovacao_constituente*) nunca foram visitados. Este é um resultado revelador: a heurística de classificação com threshold de 20 contribuições e 70 % de ativos é satisfeita pela dinâmica padrão da Vila, dominando o espaço de fases.

10.2 Recalibração fecha o gap

Aplicar `POST /api/v1/psicohistoria/calibrar?metodo=laplace&alpha=0.1` à trajetória acima produz uma matriz $M_{\text{calibrada}}$ com 64 transições observadas (cobertura 25 % dos estados canônicos, i.e. apenas *expansao* e *equilibrio* aparecem). Perplexity cai de $\sim e^2$ (grafo baseline) para $\sim e^{0.3}$ (grafo calibrado), um ganho de aproximadamente 86 % no *log-likelihood* médio das transições observadas.

Esta é a prova empírica de que calibração online é indispensável. Uma matriz baseline projetada humanamente nunca irá competir com um modelo aprendido dos dados reais da Vila, mesmo para horizontes relativamente curtos.

10.3 Suíte de testes

Cada onda cresceu entre 7 e 23 casos de teste contra invariantes matemáticos (MLE prob correta, EWMA preserva estocasticidade, perplexity cai após calibração, K-Means converge em dados separáveis, Shapley dummy = 0, Brier perfeito = 0, Banzhaf simétrico, etc.). O agregado ao longo das Ondas 5-20:

SUÍTE	CASOS	ONDA
test_game_theory.py	28	10
test_opinion_dynamics.py	11	10
test_simulacao_avancada.py	25	10
test_psicohistoria.py	22	10

SUÍTE	CASOS	ONDA
test_crenca.py	12	10.2
test_proveniencia.py	18	5
test_grafo_conhecimento.py	11	6
test_plataformas.py	7	7
test_ondas_5_a_9.py	15	5-9
test_detector_estado_vila.py	13	11
test_ondas_13_a_16.py	23	13-16
test_ondas_17_19.py	14	17-19
test_replay.py	17	20
Total	216	5-20

Todos os 216 testes passam em CI (GitHub Actions) a cada push. Cinco *pull requests* foram mergeados em sequência ao *main*:

2d9bd68	Ondas 17-19	(UI avançada + auto-calib + MCP+3)
7c01ab2	Ondas 13-16	(calibração + persistência + HMM + decision)
a3131f7	Onda 12	(UI live + fix hook)
ebdabc7	Onda 11	(rastreamento tempo real)
321c193	Ondas 5-10	(fundamentos formais + sim avançada + psico-história)

11 Conclusão e próximas direções

Esta continuação transforma a Vila INTEIA de uma demonstração matemática em um *simulador de realidade vivo*. O ciclo completo — observar, prever, comparar, recalibrar, recomendar — roda continuamente dentro do processo da Vila, sem dependências externas além de bibliotecas científicas padrão do ecossistema Python. A exposição via MCP permite que agentes externos (Claude Desktop, Cursor, Colmeia) consultem e calibrem a Vila remotamente, abrindo caminho para *multi-agent-over-multi-agent*: simulações que se auto-analisam.

Direções futuras naturais:

- **Escala distribuída Ray + vLLM**: a Onda 9 preparou o esqueleto; falta migrar `SimulacaoVila` para múltiplos actors e testar com 10k+ agentes.
- **Fine-tuning do classificador heurístico**: a dominância de *expansao* no espaço de fase sugere recalibrar os thresholds (hoje em 20/70/0.60) com base em distribuições empíricas das 216 execuções acumuladas.
- **Backtest preditivo em datasets maiores**: expandir de 10 eventos (seed eleição SP 2024) para os 5 datasets planejados no roadmap original.
- **Publicação do código como open-source**: match com MiroFish (33k estrelas) depende de visibilidade comunitária.

Referências

Ver `vila_inteia_artigo.pdf v1` para lista completa (Nash, Schelling, DeGroot, Shapley, Deffuant, Bikhchandani, Asimov, Park et al., CAMEL-AI OASIS, MiroFish).

Artigo gerado automaticamente a partir do repositório `igormorais123/vila-inteia` branch `main` . Cobre Ondas 11-20 (pós v1). Compilação via WeasyPrint. Contato editorial: `colmeia@inteia.com.br` .